



Inheritance

Problem set



گروه خونی هر فرد توسط دو `allele` تعیین می‌شود (اشکال مختلف یک ژن). سه نوع `allele` ممکن `A`، `B` و `O` هستند که هر فرد دو مورد از آن‌ها را دارد (ممکن است مشابه یا متفاوت باشند). هر یک از والدین به‌صورت تصادفی یکی از دو `allele` گروه خونی خود را به فرزندشان منتقل می‌کند؛ پس ترکیب‌های ممکن گروه خونی عبارتند از: `AB`، `AA`، `AO`، `OB`، `OA`، `OO`، `BO`، `BA` و `BB`.

برای مثال اگر یک والد گروه خونی `AO` و دیگر والد گروه خونی `BB` را داشته باشد، گروه خونی فرزند، بسته به `allele` که از هر والد دریافت کرده باشد، می‌تواند `AB` یا `OB` باشد. به همین ترتیب، اگر یک والد گروه خونی `AO` و دیگری `OB` را داشته باشند، گروه خونی فرزند `AO`، `OB`، `AB` یا `OO` می‌شود.

در فایل به اسم `inheritance.c` در فولدری به اسم `inheritance`، وراثت گروه‌های خونی را برای هر عضو یک خانواده شبیه‌سازی کنید.

در <https://cs50.dev> وارد شوید، روی ترمینال کلیک کرده و `cd` را به تنهایی اجرا کنید، ترمینال شما باید شبیه نمونه زیر باشد:

```
$
```

حالا دستور زیر را اجرا کنید تا فایل ZIP به اسم `inheritance.zip` را دانلود کنید:

```
wget https://cdn.cs50.net/2024/fall/psets/5/inheritance.zip
```

حالا با اجرا کردن دستور زیر فولدری به اسم `inheritance` بسازید:

```
unzip inheritance.zip
```

از حالا به بعد دیگر نیازی به فایل zip ندارید پس با دستور زیر به همراه Y آن را حذف کنید:

```
rm inheritance.zip
```

حالا دستور زیر را اجرا کنید تا به فولدر مورد نظر منتقل شوید:

```
cd inheritance
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
inheritance/ $
```

حالا با اجرا کردن دستور `ls` باید بتوانید فایلی به اسم `inheritance.c` را مشاهده کنید. اگر به مشکل خوردید، قدم‌های بالا را دوباره تکرار کنید.

فایل `inheritance.c` را طوری کامل کنید که یک خانواده با اندازه نسل مشخص ایجاد کند و `allele` های گروه خونی را به هر عضو خانواده اختصاص دهد. به قدیمی ترین نسل، `allele` هایی به صورت تصادفی اختصاص داده خواهد شد.

- تابع `create_family` یک عدد صحیح (`generations`) را به عنوان ورودی (تعداد نسل ها) دریافت می کند و باید برای هر عضو خانواده در تمام نسل ها، مقداری حافظه به عنوان یک `person` با استفاده از `malloc` تخصیص دهد که اشاره گری (`pointer`) به متغیر `person` در جوان ترین نسل را برگرداند.
 - برای مثال، `create_family(3)` باید یک اشاره گر به شخصی با دو والد که هر والد هم دو والد دارند را برگرداند.
 - به هر `person` باید `alleles` اختصاص داده شود. به قدیمی ترین نسل باید `allele` های تصادفی اختصاص داده شود (با فراخوانی تابع `random_allele`). نسل جوان تر باید یک `allele` (به صورت تصادفی) از هر والد به ارث ببرند.
 - به هر `person` باید `parents` اختصاص داده شود. برای قدیمی ترین نسل باید هر دو `parents` به `NULL` تنظیم شده باشد. `parents` نسل های جوان تر باید به صورت یک آرایه از دو اشاره گر باشند که هر کدام به یک والد اشاره کند.

فهم کد درون inheritance.c

به تعریف یک نوع داده به اسم `person` دقت کنید. هر `person` یک آرایه از دو `parents` دارد که هرکدام از آنها یک اشارهگر به یک ساختمان داده `person` دیگر دارند. هر `person` یک آرایه از دو `alleles` هم دارد که هرکدام یک `char` میباشد (A، B یا O).

```
// Each person has two parents and two alleles
typedef struct person
{
    struct person *parents[2];

    char alleles[2];
}
person;
```

حالا به تابع `main` نگاه کنید. این تابع با به کارگیری یک تولیدکننده اعداد تصادفی، مقادیر اولیه مورد نیاز را فراهم می‌کند؛ که بعداً برای تولید `allele`های تصادفی از آن استفاده خواهیم کرد.

```
// Seed random number generator
srandom(time(0));
```

سپس تابع `main`، تابع `create_family` را فراخوانی می‌کند تا ساخته شدن یک `person` برای یک خانواده سه نسله را شبیه‌سازی کند. (یک فرد، والدین و پدربزرگ‌ها و مادربزرگ‌ها).

```
// Create a new family with three generations
person *p = create_family(GENERATIONS);
```

سپس، ما تابع `print_family` را برای چاپ کردن اعضای آن خانواده و گروه‌های خونی آنها فراخوانی می‌کنیم.

```
// Print family tree of blood types
print_family(p, 0);
```

در آخر، تابع `main`، `free_family` را برای آزاد کردن هر حافظه‌ای که قبلاً با `malloc` تخصیص داده شده است، فراخوانی می‌کند.

```
// Free memory free_family(p);
```

کامل کردن توابع `create_family` و `free_family` بر عهده شماست!

تابع `create_family` را کامل کنید

تابع `create_family` باید یک اشاره گر به یک `person` که گروه خونی خود را از `generations` که به عنوان ورودی داده شده را برگرداند.

- ابتدا توجه کنید که این مسئله فرصت خوبی برای استفاده از بازگشت (recursion) فراهم می‌کند.
 - برای تعیین گروه خونی فرد حاضر، ابتدا باید گروه خونی والدین او را مشخص کنید.
 - برای تعیین گروه خونی آن والدین، ابتدا باید گروه خونی والدین خود آن‌ها را مشخص کنید؛ و همین‌طور ادامه دهید تا به آخرین نسلی که می‌خواهید شبیه‌سازی کنید، برسید.
- برای حل این مسئله، چندین TODO در کدهایی که در اختیارتان گذاشته شده خواهید یافت.

ابتدا باید برای یک فرد جدید حافظه اختصاص دهید. به یاد داشته باشید که می‌توانید از `malloc` برای تخصیص حافظه و از `sizeof(person)` برای به دست آوردن تعداد بایت‌های مورد نیاز استفاده کنید

```
// Allocate memory for new person person
*new_person = malloc(sizeof(person));
```

سپس، باید بررسی کنید که آیا نسل دیگری باقی مانده که باید ساخته شود یا خیر؟ با بررسی `generations > 1` می‌توانید به این سوال پاسخ دهید.

اگر `generations > 1` بود، نسل‌های بیشتری وجود دارند که باید اختصاص داده شوند. تا الان دو والد جدید با فراخوانی بازگشتی تابع `create_family` ساخته‌ایم: `parent0` و `parent1`. این تابع باید اشاره‌گرهای والدین را به `person` جدید که ساختید تنظیم کند. در آخر، هر دو `alleles` را با انتخاب تصادفی از `allele`‌های والدین به این فرد اختصاص دهید.

- به یاد داشته باشید که برای دسترسی به یک متغیر از طریق یک اشاره‌گر، می‌توانید از علامت فلش (arrow notation) استفاده کنید. برای مثال، اگر `p` یک اشاره‌گر به یک فرد است، یک اشاره‌گر به سمت اولین والد این فرد می‌تواند با `p->parents[0]` در دسترس باشد.

```
// Create two new parents for current person by recursively calling create_family
person *parent0 = create_family(generations - 1);

person *parent1 = create_family(generations - 1);

// Set parent pointers for current person
new_person->parents[0] = parent0;
new_person->parents[1] = parent1;

// Randomly assign current person's alleles based on the alleles of their parents
new_person->alleles[0] = parent0->alleles[random() % 2];
new_person->alleles[1] = parent1->alleles[random() % 2];
```

- تابع `random()` را برای تخصیص `allele`‌های تصادفی، مفید خواهید یافت. این تابع یک عدد صحیح بین `0` و `RAND_MAX` یا `32767` برمی‌گرداند. در واقع، برای تولید یک عدد شبه‌تصادفی که یا `0` باشد یا `1`، می‌توانید از عبارت `random() % 2` استفاده کنید.

بیایید تصور کنیم هیچ نسلی برای شبیه‌سازی باقی نمانده؛ یعنی `generations == 1`. در این صورت، هیچ اطلاعاتی درمورد والدی از این فرد وجود ندارد، پس هردو والد باید `NULL` تنظیم شوند و هر `allele` باید به صورت تصادفی ساخته شود.

```
// Set parent pointers to NULL
new_person->parents[0] = NULL;
new_person->parents[1] = NULL;

// Randomly assign alleles
new_person->alleles[0] = random_allele();
new_person->alleles[1] = random_allele();
```

در آخر، تابع شما باید یک اشاره‌گر به `person` اختصاص داده شده برگرداند.

```
// Return newly created person return
new_person;
```

تابع `free_family` را کامل کنید

این تابع باید یک اشاره‌گر به یک `person` را به عنوان ورودی دریافت کند، برای آن فرد حافظه آزاد کند و بعد به صورت بازگشتی حافظه تمام اجداد آن‌ها را آزاد کند.

```
// Free `p` and all ancestors of `p`. void
free_family(person *p)
{
    // Handle base case
    if (p == NULL)
    {
        return;
    }

    // Free parents recursively
    free_family(p->parents[0]);
    free_family(p->parents[1]);

    // Free child
    free(p);
}
```

- از آنجایی که این یک تابع بازگشتی است، ابتدا باید حالت پایه را مدیریت کنید. اگر ورودی تابع `NULL` باشد، چیزی برای آزاد کردن وجود ندارد، بنابراین تابع شما می‌تواند بلافاصله بازگردد.
- در غیر این صورت، باید به صورت بازگشتی، حافظه والدین فرد آزاد شود قبل از آنکه حافظه فرزند آزاد شود.

کد روبرو کاری است که باید انجام دهید:

با اجرا کردن `./inheritance` برنامه شما باید قوانین گفته شده در پیش‌زمینه را رعایت کند. فرزند باید دو `allele` که هر کدام از یک والد است را داشته باشد. والدین نیز هر کدام باید دو `allele` از هر کدام از والدینشان داشته باشند.

برای مثال، در مثال زیر، فرزند در نسل 0، یک `allele` نوع O از هر دو والدین نسل 1 دریافت کرده است. والد اول یک `allele` نوع A از پدر بزرگ اول و یک `allele` نوع O از مادر بزرگ دوم دریافت کرده است. به همین ترتیب، والد دوم یک `allele` نوع O و یک `allele` نوع B از پدر بزرگ و مادر بزرگ خود دریافت کرده است.

```
$ ./inheritance Child (Generation 0): blood type OO

Parent (Generation 1): blood type AO
    Grandparent (Generation 2): blood type OA
    Grandparent (Generation 2): blood type BO

Parent (Generation 1): blood type OB
    Grandparent (Generation 2): blood type AO
    Grandparent (Generation 2): blood type BO
```

درستی

```
check50 cs50/problems/2025/x/inheritance
```

استایل

```
style50 inheritance.c
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/inheritance
```

CS50x Iran

Harvard's Computer Science 50x Iran

